Appointment Management System

Backend: Node.js + Express + MongoDB	Frontend: React (Vite)	Default Backend: http://localhost:4000	API Prefix: /api

Table of Contents

SECTION 1 – PROJECT OVERVIEW

1.1 Project Summary

The Appointment Management System is a full-stack web application designed to digitize appointment scheduling and service management for government offices and service-based organizations. It reduces long queues and overcrowding by enabling customers to book appointments in advance, while enforcing structured scheduling rules, daily limits, and role-based access control (RBAC) to ensure security, efficiency, and accountability.

Technology Stack

- Backend: Node.js + Express.js + MongoDB (Mongoose)
- Frontend: React (Vite – latest version)

Backend Default URL

http://localhost:4000

API Base Prefix

All REST APIs are prefixed with /api. Examples:

- GET /api/customers
- POST /api/appointments

1.2 System Architecture Overview

- Frontend Layer: React + Vite, role-based dashboards, communicates via REST APIs
- Backend Layer: Express.js REST API, business logic and validation, authentication and authorization
- Database Layer: MongoDB storing Users, Customers, Services, and Appointments
- Security Layer: Passport authentication, session-based login, RBAC

1.3 User Roles Overview (RBAC)

Admin (Full Access)

The Admin has complete control over the system.

- Manage users (create, update, suspend)
- Manage customers
- Manage services
- Manage appointments (approve, update, delete)
- View analytics and reports
- Configure business rules (e.g., daily appointment limits)
- Monitor performance metrics
- Export reports (Excel/PDF)

Dashboard Access:

- Executive Overview Dashboard
- Full management panel

Staff (Moderate Access)

Staff members have operational access with restrictions.

- View assigned appointments
- Update appointment status (Pending, Approved, Completed, No Show)
- Access personal performance dashboard
- Manage bookings within assigned scope
- View daily metrics

Restrictions:

- Cannot manage system users
- Cannot change system configuration

Dashboard Access:

- Staff Dashboard
- My Dashboard
- Booking management interface

User (Basic Access)

Users have limited access.

- Create bookings
- View personal appointment history
- Track appointment status

Restrictions:

- No administrative privileges
- No analytics access
- No user management permissions

Dashboard Access:

- Personal (limited) dashboard

Role Hierarchy

Admin > Staff > User (Highest → Lowest). Each role is redirected to the correct dashboard after login.

1.4 Key Objectives

- Digitize and automate appointment scheduling
- Reduce queues and overcrowding
- Enforce daily appointment limits automatically
- Improve service efficiency and transparency
- Track performance metrics and system activity
- Ensure secure role-based access control
- Provide exportable reports (PDF / Excel)

SECTION 2 – SYSTEM ARCHITECTURE

2.1 Architecture Pattern

The system follows a Layered (Three-Tier) Architecture, separating responsibilities for maintainability, scalability, and security.

Presentation Layer (Frontend)

- Built using React (Vite – latest version)
- Provides role-based dashboards (Admin / Staff / User)
- Runs on: http://localhost:5173

Application Layer (Backend API)

- Built using Node.js and Express.js
- Uses Passport.js (Google OAuth + Session Authentication)
- Handles business logic, validation, RBAC enforcement, and API routing

API Prefix and Modules

- All API routes are prefixed with /api
- /api/users
- /api/auth
- /api/customers
- /api/services

- /api/appointments

Backend runs on: http://localhost:4000

Data Layer (Database)

- MongoDB connected via Mongoose
- Connection: await mongoose.connect(process.env.MONGO_URI);
- Initialized on startup via connectDB();
- On failure, the server exits securely using process.exit(1);

2.2 Authentication Architecture

- Google OAuth 2.0
- Session-based authentication (Passport sessions)

Authentication Flow

1. User clicks “Login with Google”
2. Google authenticates user
3. Google returns profile data
4. Passport validates the profile
5. Session is created
6. User is redirected based on role

Session Management

- express-session for persistent sessions
- passport.serializeUser() and passport.deserializeUser() for session handlers

2.3 Role-Based Access Control (RBAC)

Role hierarchy: Admin > Staff > User

- Enforced through auth.middleware.js (authentication)
- Role validation via role.middleware.js (authorization)

2.4 Request Flow (How Data Moves)

Example: Creating a Customer

7. Frontend sends POST /api/customers
8. Express Router receives request
9. Middleware validates authentication (if protected)
10. Controller processes business logic
11. Mongoose interacts with MongoDB
12. JSON response returned to the frontend

Text Flow

```
React UI → API Route (Express) → Middleware → Controller → MongoDB → JSON Response
```

2.5 Security Measures

- CORS configuration (restricted origin)
- Session security (HTTP-only cookies, expiration)
- Passport authentication
- Role-based authorization
- Protected API routes (401/403 handling)
- Environment variables stored in .env

CORS Example

```
origin: "http://localhost:5173"
credentials: true
```

2.6 System Scalability & Maintainability

Backend follows a modular structure:

```
src/
 ├── config/
 ├── controller/
 ├── middlewares/
 ├── model/
 └── routes/
```

- Clean separation of concerns
- Easier debugging
- Scalable feature expansion
- Professional backend structure

SECTION 3 – TECHNOLOGIES USED

3.1 Backend Technologies

Technology	Version	Purpose
Node.js	Runtime	Server-side JavaScript execution
Express.js	^5.2.1	REST API framework
MongoDB	Database	NoSQL data storage

Mongoose	^9.1.5	MongoDB object modeling
Passport.js	^0.7.0	Authentication middleware
Passport Google OAuth2	^2.0.0	Google login integration
Express-Session	^1.18.0	Session management

Security & Authentication

- bcryptjs – Password hashing
- jsonwebtoken (JWT) – Token-based authentication
- express-session – Persistent login sessions
- Passport.js – Authentication strategy management

Middleware & Utilities

- cors – Cross-origin resource sharing
- dotenv – Environment variable management
- multer – File uploads handling
- uuid – Unique ID generation

Development Tools

- Nodemon – Auto-restart server during development

Scripts

```
npm run dev   → nodemon server.js
npm start     → node server.js
```

3.2 Frontend Technologies

Technology	Version	Purpose
React	^19.0.0	UI Library
React DOM	^19.0.0	DOM rendering
Vite	^7.2.4	Fast build tool
React Router DOM	^7.3.0	Routing
Redux Toolkit	^2.1.2	State management

UI & Styling

- Tailwind CSS – Utility-first CSS framework
- Lucide React – Icons
- Recharts – Data visualization

API & Data Handling

- Axios – API requests
- XLSX – Excel export
- jsPDF – PDF export
- jspdf-autotable – PDF tables
- file-saver – File downloads

Frontend Scripts

```
npm run dev     → vite
npm run build   → vite build
npm run preview → vite preview
```

3.3 Why These Technologies Were Chosen

- Node.js & Express provide fast and scalable API development
- MongoDB offers flexible document-based storage
- Passport.js enables secure authentication strategies
- React ensures modular and reusable UI components
- Redux Toolkit simplifies state management
- Tailwind CSS enables rapid and clean UI styling
- Recharts provides interactive data visualization
- jsPDF & XLSX support professional reporting and exports

SECTION 4 – FOLDER STRUCTURE

4.1 Backend Directory Tree (src)

```
Backend/
 └── src/
     ├── config/
     │   ├── db.js
     │   └── passport.js
     ├── controller/
     │   ├── Customer/
     │   │   ├── appointment.controller.js
     │   │   ├── customerController.js
     │   │   └── serviceController.js
     │   ├── Dashboard/
     │   │   └── appointmentDashboard.controller.js
     │   └── User/
     │       ├── auth.controller.js
     │       └── user.controller.js
     ├── middlewares/
     │   ├── auth.middleware.js
```

```
    │   └── role.middleware.js
    ├── model/
    │   ├── Appointment.js
    │   ├── Customer.js
    │   ├── Service.js
    │   └── User.js
    └── routes/
        ├── customerRoutes/
        │   ├── customerRoutes.js
        │   └── serviceRoutes.js
        ├── user/
        │   ├── auth.routes.js
        │   └── user.routes.js
        └── appointment.routes.js
```

4.2 Folder Responsibilities

config/

- db.js – Establishes MongoDB connection (MONGO_URI)
- passport.js – Configures Passport authentication (Google OAuth + sessions)

controller/

- Contains core business logic for Customer, Dashboard, and User modules. Controllers process requests and return responses.

middlewares/

- auth.middleware.js – Ensures user is authenticated
- role.middleware.js – Ensures allowed roles can access specific routes

model/

- Defines MongoDB schemas and data models using Mongoose (User, Customer, Service, Appointment).

routes/

- Defines REST endpoints and maps them to controllers (user routes, customer/service routes, appointment routes).

4.3 Design Benefits

- Separation of Concerns: routes, controllers, models, and middleware are independent
- Scalability: features can be added by extending modules without affecting others
- Maintainability: easier debugging and cleaner codebase organization
- Security: middleware ensures authentication and role-based permissions across endpoints

SECTION 5 – ENVIRONMENT VARIABLES

Environment variables are stored in a .env file located in the backend root directory.

5.1 Required Environment Variables

```
# DATABASE
MONGO_URI=mongodb://127.0.0.1:27017/appointment_app

# AUTH / SECURITY
JWT_SECRET=your_super_secret_key
SESSION_SECRET=your_session_secret_key

# FRONTEND
FRONTEND_URL=http://localhost:5173

# GOOGLE OAUTH
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
GOOGLE_CALLBACK_URL=http://localhost:4000/api/auth/google/callback
```

5.2 Security Best Practices

- The .env file must not be committed to version control.
- Add .env to .gitignore.
- Use strong, randomly generated secrets.
- Never expose secrets in frontend code.

5.3 .env.example (Recommended for Submission)

```
MONGO_URI=
JWT_SECRET=
SESSION_SECRET=
FRONTEND_URL=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_CALLBACK_URL=
```

SECTION 6 – SERVER CONFIGURATION

This section explains how the backend server is configured, initialized, and secured.

6.1 Server Initialization

- Loads environment variables using require("dotenv").config();

- Initializes Express using const app = express();

6.2 CORS Configuration

The system allows cross-origin requests from the frontend with credentials enabled.

```
app.use(
  cors({
    origin: "http://localhost:5173",
    credentials: true,
    methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allowedHeaders: ["Content-Type", "Authorization"],
  })
);
```

6.3 Body Parsing Middleware

```
app.use(express.json());
```

6.4 Database Connection

- connectDB(); ensures database availability before processing requests.

6.5 Session Configuration

```
app.use(
  session({
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
      secure: false,  // true in production with HTTPS
      httpOnly: true,
      maxAge: 24 * 60 * 60 * 1000, // 1 day
    },
  })
);
```

6.6 Passport Initialization

```
app.use(passport.initialize());
app.use(passport.session());
```

6.7 API Routes

- app.use("/api/users", userRoutes);
- app.use("/api/auth", authRoutes);
- app.use("/api/customers", customerRoutes);
- app.use("/api/services", serviceRoutes);
- app.use("/api/appointments", appointmentRoutes);

6.8 Root Test Endpoint

```
app.get("/", (req, res) => {
  res.send("API Running");
});
```

6.9 Server Startup

```
const PORT = process.env.PORT || 4000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

6.10 Server Workflow Summary

13. Environment variables are loaded
14. Express app is initialized
15. CORS is configured
16. JSON body parser is enabled
17. Database connection is established
18. Session middleware is initialized
19. Passport authentication is enabled
20. API routes are registered
21. Server listens on the defined port

SECTION 7 – AUTHENTICATION & SECURITY

7.1 Overview

- JWT (Bearer Token) Authentication for protecting API endpoints
- Role-Based Access Control (RBAC) to restrict access based on role (Admin / Staff / User)
- Google OAuth 2.0 (Passport strategy) for login/registration
- Session support (Passport) configured for compatibility and persistence

7.2 JWT Bearer Token Authentication

Protected endpoints require a valid JWT token in request headers:

```
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

- Validates token using jwt.verify(token, process.env.JWT_SECRET)
- Normalizes role to uppercase
- Attaches decoded user to req.user
- Rejects invalid/expired tokens with 401

7.3 Admin-Only Protection

- Admin-only rule allows ADMIN (and optionally SUPERADMIN if implemented).
- Rejects others with 403.

7.4 Role-Based Authorization (RBAC)

- Authorization is enforced using allowRoles(...roles) from role.middleware.js.
- Returns 403 if user role is not in allowed roles list.

7.5 Google OAuth Authentication

- Start Google Login: GET /api/auth/google
- OAuth Callback: GET /api/auth/google/callback

7.6 User Approval (Admin Only)

- PUT /api/auth/approve/:id (requires authMiddleware + isAdmin)

7.7 Profile Completion

- PUT /api/auth/complete-profile/:id to finalize required fields (e.g., phone).

7.8 Current Logged-in User

- GET /api/auth/me (JWT required)

7.9 User Module Endpoints

- GET /api/users/me (JWT required)
- POST /api/users/login (local login if implemented)
- User management CRUD endpoints (create/list/get/update/delete) as implemented

Note (Consistency Improvement)

If your router defines /users/permanent/:id and it is mounted at /api/users, the final URL becomes /api/users/users/permanent/:id. Recommended: define /permanent/:id inside the router.

7.10 Security Summary

- JWT verification (authMiddleware)
- Admin-only protection (isAdmin)
- Flexible role checks (allowRoles)
- OAuth login support via Google
- Protected routes with clear 401 and 403 responses
- Environment-based secrets (JWT_SECRET, SESSION_SECRET, OAuth credentials)

SECTION 8 – API MODULES

Base API URL (Local): http://localhost:4000/api

Authentication (protected endpoints): Authorization: Bearer <ACCESS_TOKEN>

8.1 Customer API Module

Base Route: /api/customers

- fullName (String, required)
- phone (String, required, unique)
- email (String, optional)
- gender (String, required: MALE | FEMALE)
- isActive (Boolean, default true — used for soft delete)

Method	Endpoint	Description
GET	/api/customers/search?q=...	Search customers by name or phone (min length 2)
GET	/api/customers/search-with-status	Search customers including appointment status info
POST	/api/customers	Create a new customer
GET	/api/customers	Get all customers
GET	/api/customers/:id	Get customer by ID
PUT	/api/customers/:id	Update customer
DELETE	/api/customers/:id	Soft delete (deactivate) customer
DELETE	/api/customers/:id/permanent	Permanently delete customer (protected)

8.2 Service API Module

Base Route: /api/services

Method	Endpoint	Description
POST	/api/services	Create a service

GET	/api/services	Get all services
GET	/api/services/:id	Get service by ID
PUT	/api/services/:id	Update service
DELETE	/api/services/:id	Delete service

8.3 Appointment API Module

Base Route: /api/appointments

- Status values: PENDING, APPROVED, REJECTED, COMPLETED

Method	Endpoint	Description
POST	/api/appointments	Create appointment (daily limit handling + auto date adjustment)
GET	/api/appointments	Get pending appointments (if implemented as pending by default)
GET	/api/appointments/appointments?status=APPROVED	Get appointments by status (as currently designed)
GET	/api/appointments/:id	Get appointment by ID
PUT	/api/appointments/:id/assign-user	Assign user to appointment
PUT	/api/appointments/:id	Update appointment fields (e.g., status, notes)
DELETE	/api/appointments/:id	Cancel appointment (marks as REJECTED)
DELETE	/api/appointments/permanent/:id	Permanently delete appointment
GET	/api/appointments/my/appointments	Get my approved appointments (protected)
GET	/api/appointments/my/dashboard/analytics?month=&year=	Employee dashboard analytics (protected)

8.4 User API Module

Base Route: /api/users

Method	Endpoint	Description

POST	/api/users/login	Login user (local credentials)
GET	/api/users/me	Get current user (protected)
POST	/api/users	Create user
GET	/api/users	List users
GET	/api/users/:id	Get user by ID
PUT	/api/users/:id	Update user
DELETE	/api/users/:id	Deactivate user (soft delete)
DELETE	/api/users/users/permanent/:id	Permanent delete (admin protected; recommend simplifying)

8.5 Authentication API Module (Google OAuth)

Base Route: /api/auth

Method	Endpoint	Description
GET	/api/auth/google	Start Google login (redirects to Google OAuth)
GET	/api/auth/google/callback	OAuth callback (Passport handles + redirects)
PUT	/api/auth/approve/:id	Approve user (admin protected)
PUT	/api/auth/complete-profile/:id	Complete profile (e.g., phone)
GET	/api/auth/me	Get current user via auth route (protected)

Notes (Consistency & Improvements)

- Some routes may be public in the current implementation (e.g., Customer/Service CRUD). For production, protect them with authMiddleware and allowRoles.
- Consider simplifying duplicate user path: current /api/users/users/permanent/:id → recommended /api/users/permanent/:id

SECTION 9 – FUNCTIONALITY EXPLANATION

9.1 Customer Management

- Register new customers
- Search customers by name or phone
- Update customer information
- Deactivate (soft delete) customers
- Permanently remove customer records (restricted)

- Track customer appointment history

Validation includes:

- Required fields (full name, phone, gender)
- Unique phone number enforcement
- Gender enum validation (MALE / FEMALE)

9.2 Service Management

- Create services with unique codes
- Define service requirements (documents, identity, passport)
- Enable or disable approval workflow
- Set maximum customers per day
- Activate or deactivate services

9.3 Appointment Booking

22. Select customer
23. Select service
24. Choose appointment date
25. Submit required details

Default status on creation: PENDING

9.4 Daily Appointment Limit & Automatic Date Adjustment

- System checks how many appointments exist for the selected date (per service).
- If the daily limit is reached, the appointment is automatically shifted to the next available date.
- Repeats until an available slot is found.
- No overbooking
- Balanced service distribution
- Operational efficiency

9.5 Appointment Workflow (Approval Process)

```
PENDING → APPROVED → COMPLETED
          ↓
       REJECTED
```

- PENDING – Newly created booking awaiting review
- APPROVED – Confirmed and scheduled
- COMPLETED – Service delivered
- REJECTED – Cancelled or declined

9.6 Staff Assignment System

- Assign a user to an appointment
- Update assignment dynamically
- Track assigned appointments per staff member
- Staff can view only their assigned and approved appointments

9.7 Employee Dashboard & Analytics

Admin Dashboard

- Total bookings
- Status distribution
- Weekly activity volume
- Export (PDF / Excel)

Employee Dashboard

- Total assigned appointments
- Completed appointments
- Approved pending appointments
- No-show calculations
- Monthly analytics (filter by month and year)
- Gender distribution
- Service performance breakdown

9.8 Role-Based Access Control

- ADMIN – full system access
- STAFF – operational access (appointments & dashboard)
- USER – limited access

9.9 Authentication Mechanisms

Local Authentication

- Email & password login
- Password hashing using bcrypt
- JWT token issuance (e.g., 1-day expiry)

Google OAuth Authentication

- Google login integration via Passport
- Auto-creates user if not existing (status=PENDING)
- Requires admin approval before access
- Requires profile completion (e.g., phone)

9.10 Soft Delete & Permanent Delete Strategy

- Soft delete: sets isActive=false to preserve historical data
- Permanent delete: removes record from database; restricted to authorized roles

SECTION 10 – HOW TO RUN THE PROJECT

10.1 System Requirements

- Node.js (v18+ recommended)
- MongoDB (Local or Atlas)
- npm
- Modern browser (Chrome recommended)

10.2 Project Structure

```
Appointment-App
├── Backend   (Express + MongoDB API)
└── frontend  (React + Vite Client)
```

10.3 Backend Setup

26. Navigate to backend directory: cd Backend
27. Install dependencies: npm install
28. Create .env file and configure variables
29. Ensure MongoDB is running
30. Start backend server: npm run dev

Backend runs on: http://localhost:4000

10.4 Frontend Setup

31. Navigate to frontend directory: cd frontend
32. Install dependencies: npm install
33. Start frontend server: npm run dev

Frontend runs on: http://localhost:5173

10.5 Running the Full Application

34. Start MongoDB
35. Run backend: npm run dev
36. Run frontend: npm run dev
37. Open browser: http://localhost:5173

10.6 Default Access Flow

- Admin login (local credentials)

- Staff login
- Google OAuth login
- Role-based dashboard redirect after login

10.7 Development Mode

- Backend: nodemon server.js
- Frontend: vite (hot reload)

SECTION 11 – FUTURE IMPROVEMENTS

11.1 Online Self-Registration & Remote Booking

- Self-registration and remote booking requests
- Bookings remain PENDING until approved/rejected or reassigned

11.2 Production Deployment & Cloud Hosting

- Deploy using Render/Railway/VPS/AWS/Azure/DigitalOcean
- Add HTTPS, reverse proxy (Nginx), load balancing

11.3 Docker Containerization

- Containerize backend and frontend
- Enable consistent deployments and CI/CD pipelines

11.4 Email Notification System

- Notify on approval/rejection/reminders/account approval status
- Providers: Nodemailer, SendGrid, Mailgun

11.5 SMS Reminder Integration

- SMS confirmations and reminders
- Providers: Twilio, Africa's Talking, local gateways

11.6 Payment Integration

- Mobile money and card payments
- Payment verification before appointment approval

11.7 Audit Logging System

- Track approvals, modifications, deletions, and login history
- Improves accountability and security monitoring

11.8 Advanced Reporting & Data Export

- Custom date-range filters

- Exports: PDF, Excel, CSV
- Graph-based analytics

11.9 Appointment Time Slot System

- Introduce time slots (e.g., 09:00–09:30)
- Prevent time conflicts and improve scheduling precision

11.10 Multi-Branch Support

- Branch-specific services and staff assignment
- Location-based appointment booking

11.11 Fine-Grained Permissions

- Custom permission groups and view-only roles
- Module-level authorization rules

11.12 System Performance Optimization

- Database indexing strategies
- Caching (Redis)
- Background job processing